
UNIDAD 02

Entorno y primeros pasos

Programación en Computación · Ingeniería Electromecánica — 2.º año · UTN FR Reconquista
Longhi Pablo · Talijancic Iván

Qué vamos a ver

- 01 Las **tres herramientas** y para qué sirve cada una
- 02 La **terminal**: moverte sin el explorador de archivos
- 03 El **proyecto** de la cátedra: `npm install` y `npm run dev`
- 04 Tu **primer programa**
- 05 **Pedirle datos** al usuario

La clase de hoy es con la máquina abierta. Todo lo que veamos lo escribís vos en el momento.



PARTE 1

| Las herramientas

PREGUNTA

¿Cuál de las tres ejecuta tu programa?

Node.js · Visual Studio Code · la terminal



Tres cosas distintas

Node.js

El **intérprete**. Lee tu `.js` y lo ejecuta.

El motor

VS Code

El **editor**. Donde escribís el texto.

El taller

La terminal

Donde **das órdenes** escribiendo.

El tablero

El único imprescindible es **Node**. Sin editor podés programar; sin intérprete, no.

JavaScript no es Node.js

JAVASCRIPT

El **idioma**. Las reglas de escritura.

Un idioma no hace nada solo. Alguien tiene que hablarlo.

NODE.JS

Alguien que **habla** ese idioma y ejecuta lo que le decís.

PARTE 2

| La terminal

Siempre estás parado en una carpeta

SALIDA

```
C:\Users\ivan>
```

Ese texto te dice **dónde estás**. Todo lo que escribas se ejecuta desde ahí.

ERROR FRECUENTE

Si le pedís que corra `app.js` y en **esa** carpeta no hay ningún `app.js`, no lo encuentra — aunque el archivo exista en otro lado del disco.

Cinco comandos

QUÉ QUERÉS

WINDOWS

MACOS / LINUX

Dónde estoy

```
cd
```

```
pwd
```

Qué hay acá

```
dir
```

```
ls
```

Entrar a una carpeta

```
cd carpeta
```

```
cd carpeta
```

Volver atrás

```
cd ..
```

```
cd ..
```

Crear una carpeta

```
mkdir carpeta
```

```
mkdir carpeta
```

`..` es **"la carpeta de arriba"**. Se encadena: `cd ../..` sube dos niveles.

Armar la carpeta de la materia

SALIDA

```
cd Documents  
mkdir programacion  
cd programacion
```

Tres comandos y ya estás adentro.

La tecla `Tab` autocompleta. Escribí las primeras letras y apretá `Tab`. Es más rápido y — sobre todo — **no te equivocás tipeando**.

La flecha `↑` trae el comando anterior.

PARTE 3

| Si algo falló

Primero, verificá

SALIDA

```
node -v  
v24.19.0
```

```
npm -v  
11.6.2
```

Si las dos responden, estás listo. Si no, seguí en la próxima.

node no se reconoce

SALIDA

```
'node' no se reconoce como un comando interno o externo
```

- 01 **Cerrá la terminal y abrila de nuevo.** Resuelve la mayoría de los casos.
- 02 Si sigue: reinstalá Node dejando tildado *"Add to PATH"*.
- 03 Último recurso: reiniciá la máquina.

El **PATH** es la lista de carpetas donde el sistema busca los programas. Se lee **al abrir** la terminal: una que ya estaba abierta no ve lo que instalaste después.

PowerShell bloquea npm

SALIDA

```
npm : No se puede cargar el archivo npm.ps1  
porque la ejecución de scripts está deshabilitada
```

RÁPIDO

Usá **Símbolo del sistema** (`cmd`) en vez de PowerShell.

DEFINITIVO

Está con capturas en el **README del template**.

Le pasa a casi todos en Windows. No hiciste nada mal.

PARTE 4

| El proyecto

Abrí la carpeta correcta

× ASÍ NO

```
src/          ← abriste esta
├─ app.js
└─ prompt.js
```

VSCode no ve el `package.json` . `npm run dev` falla.

✓ ASÍ SÍ

```
mi-proyecto/ ← abrí esta
├─ package.json
└─ src/
```

La raíz: la carpeta que **contiene** el `package.json` .

Usá la terminal integrada

Ver → Terminal • **Ctrl + ñ**

Se abre ya parada en la carpeta del proyecto.

Te ahorra el **cd** y te garantiza que estás en el lugar correcto.

USALA SIEMPRE.

Qué hay adentro

ARCHIVO

QUÉ ES

package.json

La ficha técnica: dependencias y comandos

src/app.js

👉 **Acá escribís vos**

src/prompt.js

Leer del teclado. **No lo toques**

eslint.config.mjs

Las reglas de estilo con las que se corrige

node_modules/

Las dependencias. **No se toca**

Dos comandos, y cuándo

terminal

```
npm install    # una sola vez, al empezar el proyecto
npm run dev    # cada vez que querés correr tu programa
```

INSTALL

Descarga las dependencias a `node_modules/`. Necesita internet.

¿No ves `node_modules`? Corrélo.

DEV

Corre `src/app.js` y **queda mirando** el archivo: guardás y se re-ejecuta solo.

Cortar: `Ctrl + C`

PARTE 5

| Tu primer programa

Una sola línea

src/app.js

```
console.log('Hola mundo')
```

SALIDA

```
Hola mundo
```

Eso es un programa. Una instrucción, y la máquina la ejecutó.

Cómo se lee

src/app.js

```
console.log('Hola mundo')  
//   ↑           ↑  
//   |           └ el argumento: qué mostrar  
//   └ la orden: "escribí esto en la consola"
```

Los **paréntesis** son obligatorios. Las **comillas** marcan dónde empieza y termina el texto.

Varios argumentos

src/app.js

```
console.log('Tensión:', 220, 'V')
```

SALIDA

```
Tensión: 220 V
```

Separados por comas. Se imprimen en la misma línea.

Meter valores dentro del texto

× CON COMAS

```
console.log('Tensión:',  
            tension, 'V')
```

Se corta el texto en pedazos.

✓ CON BACKTICKS

```
console.log(  
    `Tensión: ${tension} V`)
```

Se lee de corrido.

ERROR FRECUENTE

El **backtick** ``` no es una comilla simple `'`. Está a la izquierda del `1`. Con comilla simple, el `${tension}` se imprime tal cual.

Comentarios

src/app.js

```
// Un comentario de una línea

/*
  Un comentario
  de varias líneas
*/

console.log('Esto sí se ejecuta')
```

La computadora los ignora. Son para el humano que lea el código — casi siempre, vos dentro de tres semanas.

PARTE 6

| Guardar y pedir datos

Ponerle nombre a un dato

src/app.js

```
const tension = 220
const corriente = 5

console.log(`Potencia: ${tension * corriente} W`)
```

SALIDA

```
Potencia: 1100 W
```

Escribir `tension` es lo mismo que escribir `220`. En la **unidad 03** vemos `let`, `const` y `var` en detalle.

PREGUNTA

¿Y si quiero la potencia de otro motor?

Con los valores escritos en el código, hay que editar el programa cada vez.



Pedirle datos al usuario

src/app.js

```
import { prompt } from './prompt.js'

const nombre = prompt('¿Cómo te llamas? ')
console.log(`Hola ${nombre}, bienvenido al curso`)
```

SALIDA

```
¿Cómo te llamas? Ana
Hola Ana, bienvenido al curso
```

El `import` va **siempre al principio** del archivo. El programa se detiene y espera el `Enter`.

PREGUNTA

Leo 20 y 5, los sumo. ¿Qué imprime?

```
console.log(a + b)
```



No da 25

src/app.js

```
const a = prompt('Primer número: ')\nconst b = prompt('Segundo número: ')\n\nconsole.log(a + b)
```

SALIDA

```
Primer número: 20\nSegundo número: 5\n205
```

ERROR FRECUENTE

`prompt()` devuelve **siempre texto**. Y para el texto, `+` no suma: **pega**.

Convertir a número

src/app.js

```
const a = parseInt(prompt('Primer número: '))  
const b = parseInt(prompt('Segundo número: '))  
  
console.log(a + b)
```

SALIDA

```
Primer número: 20  
Segundo número: 5  
25
```

Si el dato es un número y lo vas a usar para calcular: **envóvelo**. Siempre.

parseInt o parseFloat

FUNCIÓN	CONVIERTE A	'21.5' DA
<code>parseInt()</code>	Entero	<code>21</code>
<code>parseFloat()</code>	Con decimales	<code>21.5</code>

ERROR FRECUENTE

`parseInt('21.5')` te devuelve `21` y **perdés el decimal sin ningún aviso**. Una temperatura, una tensión o una longitud rara vez son enteras: ante la duda, `parseFloat`.

PREGUNTA

21,7 °C a Kelvin es sumarle
273,15. ¿Qué imprime?

Debería dar 294,85.



Los decimales que no pediste

src/app.js

```
const celsius = 21.7  
console.log(`${celsius + 273.15} K`)
```

SALIDA

```
294.84999999999997 K
```

No es un error tuyo: es $0.1 + 0.2$ de la unidad 01. Hay decimales que en binario **no tienen representación exacta**.

.toFixed()

src/app.js

```
const celsius = 21.7  
const kelvin = celsius + 273.15  
  
console.log(`${kelvin.toFixed(2)} K`)
```

SALIDA

294.85 K

Calculá con los números completos. Redondeá **sólo al mostrar**.

Todo junto

src/app.js

```
import { prompt } from './prompt.js'

const motor = prompt('Identificación del motor: ')
const tension = parseFloat(prompt('Tensión [V]: '))
const corriente = parseFloat(prompt('Corriente [A]: '))

const potencia = tension * corriente
console.log(`Motor ${motor}: ${potencia.toFixed(2)} W`)
```

SALIDA

```
Identificación del motor: M-14
Tensión [V]: 380
Corriente [A]: 4.2
Motor M-14: 1596.00 W
```

PARTE 7

| Cuando algo falla

Un error no es un castigo

SALIDA

```
ReferenceError: tencion is not defined  
    at file:///Users/ivan/mi-proyecto/src/app.js:4:13
```

Te dice **qué pasó**, en **qué archivo** y en **qué línea**.

Leé la **primera línea** y buscá el **número de línea**. El resto casi siempre es ruido interno de Node.

Los cuatro que vas a ver hoy

MENSAJE

QUÉ SIGNIFICA

SyntaxError

Está **mal escrito**: falta un paréntesis, llave o comilla

ReferenceError: x is not defined

Nombraste algo que **no existe**. Revisá cómo lo escribiste

Cannot find module

Estás en la **carpeta equivocada**

Cannot find package 'readline-sync'

Falta `npm install`

Lo que hay que llevarse

- 01 El que **ejecuta** es Node. VSCode sólo te ayuda a escribir.
- 02 En la terminal **siempre estás parado en una carpeta**. Usá la integrada de VSCode.
- 03 Abrí **la raíz**: la carpeta que contiene el `package.json`.
- 04 `npm install` una vez · `npm run dev` siempre.
- 05 `prompt()` devuelve **texto**. Convertilo con `parseFloat()` y mostralo con `.toFixed(2)`.

Para la próxima

Dejá el entorno andando

`npm run dev` tiene que correr sin errores.

Hacé el TP

Seis problemas. Es **la** práctica que necesitás para que la unidad 03 no se te haga cuesta arriba.

Si algo no te anduvo, **traelo anotado**. No te quedes trabado una semana.

PRÓXIMA CLASE • UNIDAD 03

Variables y operadores

`let`, `const` y `var` • Tipos de datos • Operadores